SSX Silicon 1: Step by Step Operator Guide

Comprehensive User Manual for Non-Technical Evaluators and Operators

Author: Samin Sami (SS) **Architecture:** SSX-32 (32-Bit Dual Engine) **Status:** Formally Verified (100% Pass)

Step by step guide for SSX Silicon 1

Developer: Samin Sami (SS)
Project: SSX Silicon 1 32-Bit Processor in Google Sheets

This guide walks you through opening, running, and inspecting the SSX Silicon 1 processor in Google Sheets. You do not need any background in programming or electronics to follow these steps.

What this project is

SSX Silicon 1 is a complete 32-bit computer processor built directly inside a spreadsheet. It contains 32 internal registers, an arithmetic logic unit for math and comparisons, 4,096 memory words, and a 32 by 32 pixel color display screen.

When you run a program, the processor reads instructions from memory, calculates results, updates numbers in real time, and draws pixels on the screen. Everything you see is calculated live using real processor logic. There are no prerecorded animations or fake mockups.

Step 1: Open Google Sheets and import the file

1. Open your web browser (Google Chrome, Firefox, or Safari) and navigate to https://sheets.google.com.
2. Sign in to your Google account if prompted.
3. Click on the Blank spreadsheet button to start a new file.
4. In the top menu, click on File, then click on Import.
5. In the window that appears, click the Upload tab.
6. Drag and drop the file named `SSX_Silicon_1_CPU.xlsx` into the window, or click Browse to select it from your computer.
7. Under the Import location options, select Replace spreadsheet.
8. Click the Import data button.

Google Sheets will load the workbook. You will see eight tabs along the bottom:

- `CONTROL_DASHBOARD`: The main control room showing the clock, program counter, and status lights.
- `REGISTERS_32`: The 32 internal storage registers (R0 through R31) holding active numbers.

- `ALU_32` : The math unit that performs addition, multiplication, division, and logic tests.
- `RAM_MEMORY` : The primary working memory holding 4,096 32-bit numbers.
- `ROM_STORAGE` : The storage grid where assembled programs sit before running.
- `DISPLAY_SCREEN` : A 32 by 32 color grid that acts as the computer monitor.
- `INSTRUCTION_SET` : A reference sheet listing the 36 processor instructions.
- `USER_MANUAL` : A quick reference card for common commands.

Step 2: Install the Apps Script engine (takes one minute)

The workbook includes a Google Apps Script file that lets you run the CPU automatically and control it with convenient menu buttons.

1. In the top menu of Google Sheets, click Extensions, then click Apps Script.
2. A new browser tab opens showing a code editor.
3. If there is any default code like `function myFunction() {}`, select it and press Delete to clear the editor.
4. On your computer, open the file `scripts/SSX_Silicon_Engine.js` using any text editor (such as Notepad, TextEdit, or VS Code).
5. Select all the text (Ctrl+A on Windows, Cmd+A on Mac) and copy it (Ctrl+C or Cmd+C).
6. Paste the code into the Google Apps Script editor window.
7. Click the Save icon (the small floppy disk) at the top of the editor, or press Ctrl+S.
8. Close the Apps Script browser tab and return to your Google Sheets tab.
9. Refresh the Google Sheets tab in your browser (press F5 or click the reload button).

Once the page reloads, wait about five seconds. A new menu item titled `SSX Silicon 1` will appear in the top menu bar, right next to Help.

Step 3: Run the hardware self-test

This test verifies that all 32 registers, the ALU math operations, the memory read/write cycles, and the stack work properly.

1. Click on the tab named `CONTROL_DASHBOARD` at the bottom of the screen.
2. In the top menu bar, click on `SSX Silicon 1`.
3. Move your mouse over `Load Program`, then click on `1. Hardware Self-Test`.
4. A notification toast appears at the bottom right corner of the sheet saying the program has loaded into ROM.
5. Click on `SSX Silicon 1` in the top menu again, and click `Run Program (Continuous)`.
6. Watch the screen. The Program Counter (cell D6) will advance from 0x0000 through the test routine, and the Cycle Counter (cell D8) will count up to 77.
7. Look at cell B23 on the dashboard, titled `HARDWARE MMIO PORT 0x2003 (STATUS MONITOR)`.
8. The value in cell B23 displays `0xCAFEBABE`.

9. Click on the `REGISTERS_32` tab. Cell C5 (register R1) also holds `0xCAFEBABE`.

If any test had failed, the CPU would have stopped with an error code starting with `0xEE`. The signature `0xCAFEBABE` proves that every mathematical operation, comparison, branch, and memory store passed verification.

Step 4: Calculate real Fibonacci numbers

This program computes the first 47 numbers in the Fibonacci sequence in 32-bit math and writes them into memory.

1. Return to the `CONTROL_DASHBOARD` tab.
2. In the top menu, click `SSX Silicon 1`, then `Reset CPU (PC = 0)`.
3. In the top menu, click `SSX Silicon 1`, hover over `Load Program`, and click `2. 32-Bit Fibonacci`.
4. Click `SSX Silicon 1` again, and click `Run Program (Continuous)`.
5. The CPU runs for 419 cycles and stops at the HLT instruction.
6. Look at cell B23 on the dashboard or cell C5 in the `REGISTERS_32` tab.
7. Register R1 displays `0x6D73E55F`, which is the hexadecimal representation of 1,836,311,903 (the 46th Fibonacci number).
8. Click on the `RAM_MEMORY` tab. You can see the computed sequence stored across the memory cells starting at row 2.

Step 5: Run the Sieve of Eratosthenes prime finder

This test finds all prime numbers up to 100 using a memory sieve algorithm.

1. Click on `CONTROL_DASHBOARD`.
2. In the top menu, click `SSX Silicon 1`, then `Reset CPU (PC = 0)`.
3. In the top menu, click `SSX Silicon 1`, hover over `Load Program`, and click `3. Prime Sieve`.
4. Click `SSX Silicon 1` and select `Run Program (Continuous)`.
5. The CPU completes 2,264 cycles.
6. Cell B23 and Register R1 display `0x00000019`. In decimal, 0x19 is 25. There are exactly 25 prime numbers between 1 and 100.

Step 6: Watch the 32 by 32 color screen draw in real time

The processor includes a dedicated 1,024 word video memory buffer (VRAM) that directly controls the 32 by 32 pixel grid on the `DISPLAY_SCREEN` tab.

1. Click on `CONTROL_DASHBOARD`.
2. In the top menu, click `SSX Silicon 1`, then `Reset CPU (PC = 0)`.
3. In the top menu, click `SSX Silicon 1`, hover over `Load Program`, and click `4. 32x32 Graphics Rendering`.
4. Click on the `DISPLAY_SCREEN` tab at the bottom of the workbook so you can see the screen.

5. In the top menu, click `SSX Silicon 1`, and click `Run Program (Continuous)`.
6. Watch the grid. The CPU clears the display to black, draws borders in neon blue, plots crosshairs in cyber amber, and renders a central crest in crimson and gold.
7. Exactly 232 pixels light up with colors according to the 16-color palette.

Step 7: Step through instructions one by one

If you want to observe how a computer processor thinks step by step:

1. Click on `CONTROL_DASHBOARD`.
2. In the top menu, click `SSX Silicon 1`, then `Reset CPU (PC = 0)`.
3. Click `SSX Silicon 1`, hover over `Load Program`, and choose any program (such as Fibonacci).
4. Instead of clicking Run, click `Step Clock (1 Cycle)`.
5. Observe cell D6 (Program Counter). It increments.
6. Observe cell B16 (Raw Instruction). It displays the 32-bit machine code word currently fetched.
7. Observe cell B19 (Instruction Bus). It shows human-readable assembly text, such as `LOADI R10, R0, R0 [#4096]`.
8. Observe cell D10 and F10 (Status Flags). These show whether the previous operation was zero, negative, or caused an overflow.
9. Click `Step Clock (1 Cycle)` again to watch the next instruction execute.

How to run using spreadsheet formulas without Apps Script

You can also run the processor using Google Sheets iterative formula mode without any scripts:

1. Click `File` in the top menu, then click `Settings`.
2. Click the `Calculation` tab.
3. Turn `Iterative calculation` to `On`.
4. Set `Max number of iterations` to `1`.
5. Click `Save settings`.
6. Go to `CONTROL_DASHBOARD`.
7. Click on cell `B4` (MASTER CLOCK).
8. Change its value from `0` to `1`, or press the Delete key and then Enter.
9. Each time the cell recalculates, the circular formula loop in the spreadsheet advances the Program Counter and updates the registers.

Summary of golden test values

You can confirm that the system is operating without any errors by checking these output values:

Test Program	Execution Cycles	Expected Output in R1	What it proves
Hardware Self-Test	77 cycles	`0xCAFEBABE`	All 36 opcodes, registers, and memory work
32-Bit Fibonacci	419 cycles	`1,836,311,903` (`0x6D73E55F`)	High-range arithmetic and loop branching work
Prime Sieve	2,264 cycles	`25` (`0x00000019`)	Memory arrays and conditional jumps work
3x3 Matrix Multiply	671 cycles	`[30, 24, 18, 84, 69, 54, 138, 114, 90]`	Vector dot-products and memory loads work
Graphics Engine	1,034 cycles	232 active colored pixels	Memory-mapped I/O and display hardware work